Main Properties for Developers

This page describes issues relevant for all developers when adding new fields into class `MainProperties` or its nested properties.

Notification

✅ Whenever you introduce a new field, you need to inform one of the deployers. The current set of deployers is defined by mailing list deployer-dstores@exedio.com. Please do provide information about values needed for these fields on QA and production, if you know already.

✅ Verify changes to the MainProperties with the `MainPropertiesTest`.

You don't need to (and must not) make any changes to any installed systems.

Default Values

For each new field you have to decide, whether the field should have a default value of not.

- Good candidates for setting a default are: connection timeouts, test mode flags.
- Good candidates for **not** setting a default are: host names, ip addresses, user names and especially passwords.

⚠️ Defaults must be *conservative*. This means, both the probability of accidentally leaving the default in place, and the damage caused by this should be minimized. For instance

- the test flag of a payment provider should **enable** the test mode by default, but in contrast:
- the debugging mode (i.e. wicket) should be **disabled** by default.

If the value of a property will probably the same for all systems (production, QA, developer), than that value should be the default.

Please note, that fields of type:

- `boolean` and `int` cannot be specified without defaults.
- `String` can be specified with or without defaults.
- `File` can be specified without defaults only.

For any new field without a default you need two entries in `project.properties` with prefix "tomcat." and "test.".

No Change of Semantics

❌ You must never change the semantics of any field. Examples of such forbidden change are:

1. There is a field `connectionTimeout`, which specifies a certain timeout in seconds. If you change it to hold milliseconds instead, this is a forbidden change of semantics. Instead you must replace field `connectionTimeout` by a new field `connectionTimeoutMilliseconds`. Of course you may do this by just changing the key of field `connectionTimeout`.
2. There is a field `rest.url` holding the url of a remote rest service. There is a newer version of this service available on a new url. The newer version uses a new schema, which is not backward compatible to the old service. Using field `rest.url` for connection to the new service is a forbidden change of semantics. Instead you must replace field `rest.url` by a new field `rest.urlNew` or similar.

Changing semantics of property fields will cause severe problems for deployers keeping track of changes needed for installed systems. Therefore please do not do it.

Reuse of MainProperties.get()

When reusing the result of `MainProperties.get()`, you must reuse it correctly. You may reuse it either too much or not enough. This does not only apply to the instance of `MainProperties` itself, but to any other result derived from it.

Required Reuse

You must reuse the result for avoiding inconsistencies. For example the following code is wrong:

```
createConnection(
  MainProperties.get().username, ✗
  MainProperties.get().password);
```

because the two calls to might return different instances when the properties are changed via the PropertiesServlet.

Do this instead:

```
MainProperties props = MainProperties.get(); ✓
createConnection(
  props.username,
  props.password);
```

Forbidden Reuse

If you reuse it for too long, this inhibits the ability to change properties via the PropertiesServlet. In general you should not reuse longer that a request or a transaction. ✗ Never store it to any static field.